Daily Paper Review: Relevance Does Not Imply Applicability: Experience Activation for Personal GUI Agents

Auto-generated Report

October 14, 2026

1 Paper Summary

1.1 Problem and Motivation

Personalized GUI agents that accumulate interaction history should theoretically improve over time, but current retrieval-based approaches implicitly assume that *task-level relevance implies decision-time applicability.* Zhang et al. (arXiv:2609.33304) demonstrate that this assumption is empirically false. They decompose agent performance into **opening decisions** (∼10% of steps: choosing which app to launch) and **within-trajectory steps** (∼90%: navigating screens, filling forms, etc.), and show a striking result: four existing baselines (Recent, Retrieve, LLM-UM, HIM-Agent) raise opening-step success from 28.3% to 62.3%, but for the remaining 7,140 within-trajectory actions, *all four methods are statistically indistinguishable from using no history at all* (95% CIs spanning zero). The problem: appending whole retrieved trajectories (730–844 tokens) floods the context with information irrelevant to the current screen state, causing context dilution, stale references, and 3–6× token waste without improving action accuracy.

The paper introduces the distinction between **relevance** (task-level: "does this trajectory concern a similar task?") and **applicability** (decision-time: "does this specific historical action fit the current screen, temporal context, and scenario?"). This distinction is central to continual learning for agents: as experience accumulates, the retrieval problem shifts from finding *what is related* to activating *what is actionable now.*

1.2 Method

ExpActivator is a **training-free** two-level activation framework operating over frozen backbones and encoders. For user u with interaction history $\mathcal{M}_u = \{r_i\}$, each record $r_i = (g_i, p_i, c_i, \tau_i, \xi_i)$ contains a goal, application, scenario category (1 of 13), timestamp, and trajectory of (screenshot, action) pairs.

Unified Activation. For granularity $d \in \{\text{step}, \text{task}\}$:

$$e_u^{(d)}(x) = \Gamma_d \left(\underset{b \in \mathcal{B}_u^{(d)}(x)}{\arg\max} \kappa_d(x, b),\ x \right) \tag{1}$$

where $\mathcal{B}_u^{(d)}$ is the candidate set, κ_d is the compatibility function, and Γ_d extracts the actionable output.

Step-Level Activation. A two-stage pipeline runs at every observation:

Stage 1 (once per task): Retrieve top-K_ξ=8 trajectories by goal-embedding similarity $\alpha_i(q) = f(q)^\top f(\rho(g_i))$, where f is paraphrase-multilingual-MiniLM-L12-v2 and ρ strips leading application phrases.

Stage 2 (every step): Compute a centered block-wise visual representation of the current screen:

$$\phi(s) = \frac{1}{\sqrt{B}}\big[\nu(v_1(s) - \mu_1);\ \ldots;\ \nu(v_B(s) - \mu_B)\big] \tag{2}$$

where B=4 spatial quadrants (5 for MAI-UI with a global block), $v_k(s)$ are pooled visual features from the frozen backbone, μ_k are block-wise means from calibration images, and ν is L2 normalization. The best-matching historical step is selected jointly across all candidate trajectories:

$$(i_t^*, j_t^*) = \underset{i \in \mathcal{C}_u(q),\, 0 \le j < T_i}{\arg\max} \phi(s_t)^\top \phi(s_{i,j}) \tag{3}$$

The matched action a_{i^*,j^*} is serialized as a single reference for the policy. Critically, this reference is **recomputed at every observation**, allowing it to migrate across different historical trajectories as the task progresses through different screens.

Task-Level Activation (Proactive Suggestion). For unprompted suggestion of user routines, ExpActivator identifies routine neighborhoods via intent-embedding similarity (η=0.94 threshold), then computes **routine support** combining scenario agreement and temporal Gaussian weighting:

$$M_j(x) = \sum_{i \in \mathcal{R}_j(x)} \mathbb{1}[c_i = c_x] \cdot \exp\left(-\frac{d_{24}(\omega_i, \omega_x)^2}{2\sigma^2}\right) \tag{4}$$

where $d_{24}(a,b) = \min(|a-b|, 24-|a-b|)$ is the circular hour distance and σ=1 hour. Hard eligibility thresholds ($n_{\min}$=12 total, n_{ctx}=4 same-scenario, $M_j \ge 0.25$) gate whether to suggest at all. Eligible routines are ranked by support-per-occurrence $\kappa_{\text{task}} = M_j / n_j^\gamma$ (γ=1). If no routine qualifies, the system **abstains**—a critical capability that baselines lack.

1.3 Results

Experiments use the AndroidIntent benchmark (82 users, 135 apps, 15,966 historical records, 775 evaluation episodes with 7,915 steps, 315 proactive states) across four frozen backbones: Qwen3-VL-8B, MAI-UI-8B, Qwen3.5-9B, GUI-Owl-1.5-8B.

Personalized Execution: On Qwen3-VL-8B, ExpActivator achieves **30.9% SSR** (step success rate) vs. 22.6% for the best baseline (Retrieve/HIM-Agent) and 19.4% for no history—a +37% relative improvement over baselines. CSR (complete success rate) reaches **47.9%** vs. 41.4%. Gains are consistent across all four backbones. All open models with ExpActivator surpass proprietary QwenVL-Max in CSR by $\sim$1.6×.

The critical within-trajectory finding: ExpActivator achieves **+8.24 pp** within-trajectory SSR improvement with 95% CI [+6.51, +10.16] (entirely above zero), while all four baselines have CIs spanning zero. ExpActivator uses only **144.6 history tokens/step**—5× fewer than baselines (730–844 tokens)—while delivering the only statistically significant improvement.

Proactive Suggestion: ExpActivator achieves **MCC 42.7–45.2** across backbones, vs. 14.1–22.0 for the best baseline (HIM-Agent). Precision reaches 82.6–86.5% (vs. 68.4–82.4%). Baselines without abstention capability achieve MCC $\approx$ 0, performing indistinguishably from an "always suggest" rule.

Cross-trajectory activation: 92% of episodes draw references from $\ge$2 past trajectories (avg. 3.4), with references switching to a different trajectory on 55% of consecutive steps—confirming that no single trajectory is "the right one" throughout.

1.4 Limitations

(1) **History size dependence**: Task-level activation requires $n_{\min}$=12 routine repetitions, limiting coverage for users with sparse histories or novel habits. (2) **No learning from failures**:

The framework does not update based on whether activated experiences led to successful or failed actions—there is no feedback loop. (3) **Frozen backbone assumption**: The method relies entirely on pretrained visual and text encoders; novel app interfaces poorly represented in pretraining may yield weak screen matching. (4) **Single benchmark**: All evaluation uses AndroidIntent; generalization to iOS, desktop, or web platforms is not demonstrated. (5) **Conservative abstention**: The hard thresholds ($n_{\min}$=12, n_{ctx}=4) make proactive suggestion unavailable for recently acquired habits.

1.5 Relevance to Research

ExpActivator directly addresses Topic T3 (continual learning for LLM agents), specifically the challenge of *how* accumulated experience should inform future decisions. The relevance-vs-applicability distinction connects to EvoEmbedding (Review #77), which proposed evolvable embeddings for long-context agentic memory—ExpActivator solves the complementary problem of *when and how* to activate retrieved memories, not just how to store them. The per-step screen matching resonates with Remember When It Matters (Review #91), which introduced proactive memory triggers for long-horizon agents; ExpActivator provides the missing *applicability filter* that determines whether a triggered memory is actionable at the current state. The training-free, retrieval-based approach contrasts with Macaron-V1 (Review #112), which used mixture-of-LoRA for continual learning—ExpActivator achieves strong gains without any parameter updates, suggesting that the activation strategy may matter more than the representation for experience-driven agents. The temporal Gaussian weighting in routine support (d_{24} circular distance) connects to the phase-binned positional encoding in TinyCast (Review #119), which similarly exploited periodic temporal structure (FFT-detected periodicity) without learned parameters.

2 Follow-up Research Directions

2.1 Direction 1: Feedback-Driven Activation Refinement via Outcome-Weighted Screen Matching

Gap: ExpActivator treats all historical actions as equally trustworthy regardless of whether they led to task success or failure. The screen matching score $\phi(s_t)^\top \phi(s_{i,j})$ depends only on visual similarity, not on the outcome quality of the matched action. A high visual match to a historically *failed* action is worse than a weaker visual match to a *successful* one.

Approach: Augment the step-level compatibility with an outcome-weighted term:

$$\kappa'_{\text{step}}(x_t, (i,j)) = \phi(s_t)^\top \phi(s_{i,j}) + \lambda \cdot w_{\text{out}}(i)$$

where $w_{\text{out}}(i)$ is derived from the trajectory-level outcome (binary success/failure) or, more granularly, from step-level credit assignment. For credit assignment without additional labeling, adopt the token-level credit approaches from DelTA (Review #57) or CoRT (Review #100)—both assign per-step rewards by counterfactual analysis of action contributions to task completion. The outcome weight is updated online as new trajectories complete: successful trajectories increase their constituent actions' weights; failed trajectories decrease them. This introduces a feedback loop absent from ExpActivator while remaining training-free (only scalar weights are updated, not model parameters). Connect to the Escaping the Self-Confirmation Trap framework (Review #78), which used execute-distill-verify to prevent agents from reinforcing incorrect experiences.

Feasibility: High. Outcome labeling (success/failure per trajectory) is already available in the AndroidIntent benchmark. Step-level credit assignment via counterfactual analysis adds one additional forward pass per completed trajectory. The main risk is that outcome-weighted

retrieval may reduce exploration diversity by over-favoring historically successful but suboptimal actions.

2.2 Direction 2: Diffusion-Based Experience Interpolation for Sparse-History Agents

Gap: ExpActivator's task-level activation requires $n_{\min}$=12 routine repetitions before suggesting anything, and step-level matching fails when no historical trajectory contains a screen visually similar to the current one. Users with sparse histories (new users, novel apps) cannot benefit from the activation framework. The fundamental problem is that experience activation is limited to *exact retrieval*—it cannot interpolate between experiences to synthesize applicable actions for unseen situations.

Approach: Use a diffusion language model (connecting T3 with T4) to generate **synthetic experience interpolations**. Given two retrieved trajectories with partial screen matches, use a dLLM to generate plausible intermediate actions by conditioning on: (a) the current screen embedding, (b) the two flanking historical actions, and (c) the task goal. The dLLM's iterative denoising naturally handles the uncertainty of interpolation—early denoising steps commit to action type (click vs. scroll vs. type), while late steps refine coordinates and text. This extends ExpActivator's retrieval-only paradigm to a retrieval-augmented generation paradigm for agent actions. The block-diffusion architecture of Fast-dLLM v2 (whose quantized inference SoloQ enables, Review #123) is particularly suited: each block generates one action, and the bidirectional attention naturally conditions on both flanking historical contexts. Lower the $n_{\min}$ threshold proportionally to the interpolation model's confidence, enabling proactive suggestions for routines with as few as 3–4 historical repetitions.

Feasibility: Medium. Requires fine-tuning a dLLM on action generation conditioned on screen embeddings, which is a non-trivial but well-defined task. The main challenge is evaluating interpolated action quality—synthetic actions have no ground-truth outcome, so validation requires online deployment or simulation. The block-diffusion architecture's compatibility with SoloQ W4A4 quantization (Review #123) makes inference efficient enough for real-time generation.

2.3 Direction 3: Cross-User Experience Transfer via Privacy-Preserving Activation Sharing

Gap: ExpActivator operates strictly within each user's own history ($\mathcal{M}_u$). When a user encounters a novel app or task with no personal history, the system has nothing to activate. Meanwhile, other users may have extensive experience with that same app. The challenge is enabling cross-user experience transfer without exposing private interaction data (screen contents, typed text, browsing history).

Approach: Share only the **centered screen representations** $\phi(s)$ and **action templates** (action type + normalized coordinates, with all typed text replaced by placeholder tokens) across users, creating a shared activation index. The centering operation $(v_k(s) - \mu_k)$ already removes mean visual features, and L2 normalization discards magnitude—the resulting representations are abstract enough to resist screen content reconstruction while preserving layout similarity for matching. At query time, a user's step-level activation searches both their personal index and the shared community index, with a mixing coefficient β that decays as personal history grows: $\beta = \exp(-|\mathcal{M}_u|/\tau)$, favoring personal experience as it accumulates (connecting to the continual learning principle of gradually shifting from community to personal knowledge). This relates to Co-RL (Review #118), where cyclic peer pseudo-labels enabled label-free learning across agents—here, cross-user activation sharing provides "experience pseudo-labels" for novel tasks.

Feasibility: Medium. The privacy analysis requires formal guarantees (differential privacy bounds on screen reconstruction from $\phi(s)$). The centered, normalized block representations likely provide practical privacy through information bottlenecking (reducing 1024×1024 screenshots to $4 \times d$-dimensional normalized vectors), but formal analysis is needed. Scalability of the shared index (82 users $\times$ 15,966 records in AndroidIntent) is manageable with approximate nearest-neighbor search.

3 Conclusion

ExpActivator establishes a fundamental principle for experience-driven agents: *experience pays where it is activated, not where it is appended.* By replacing wholesale trajectory retrieval with per-step screen matching and temporally-gated routine activation, the method delivers the first statistically significant within-trajectory improvement (+8.24 pp, CI entirely above zero) while using 5$\times$ fewer history tokens than baselines. The training-free design—relying entirely on frozen encoders, centered visual representations, and hard eligibility thresholds—demonstrates that the activation strategy matters more than the representation for turning accumulated experience into actionable guidance. The relevance-vs-applicability distinction provides a concrete framework for future continual learning systems: as agents accumulate more experience, the challenge shifts from retrieval recall to activation precision, and methods that conflate the two will see diminishing returns from growing memory stores.

References

[1] F. Zhang, X. Wang, Z. Li, J. Zhang, L. Huang, and W. Y. B. Lim. Relevance Does Not Imply Applicability: Experience Activation for Personal GUI Agents. *arXiv preprint arXiv:2609.33304*, 2026.

[2] Z. Wang et al. HIM-Agent: Hierarchical Memory Agent for Personalized GUI Interaction. *arXiv preprint*, 2024.

[3] Y. Li et al. LLM-UM: User Modeling via Large Language Models for Personal Agents. *arXiv preprint*, 2025.

[4] Y. Lu et al. EvoEmbedding: Evolvable Representations for Long-Context Retrieval and Agentic Memory. *arXiv preprint arXiv:2606.21649*, 2026.

[5] S. Wu et al. Remember When It Matters: Proactive Memory Agent for Long-Horizon Agents. *arXiv preprint arXiv:2607.08716*, 2026.